

Porting CMS Applications to WebDAV and Cocoon

Gianugo Rabellino g.rabellino@pro-netics.com

http://www.pro-netics.com - http://www.orixo.com

open source system integration

(pro-netics)



ApacheCon 2003



Who am I

- Involved in Open Source since 1993
- Involved in Cocoon since 1999, committer since 2001
- CTO of Pro-netics, an Orixo alliance company
- Former lawyer (well, almost…)
- Italian
    - Bear with my funny accent and lousy English
    - Ask questions loud and clear, please



Introducing the session

- Our personal view on the whole CMS business
- WebDAV
- Cocoon
- WebDAV *and* Cocoon
- Performance issues: ESI to the rescue
- Putting it all together: a migration strategy

CMS lessons learned (the hard way):

- No “one size fits all”
- CMS or SMT (Site Management Tool)?
- RDBMs are suboptimal:
  - Content needs hierarchies (self-referencing foreign keys?)
  - Content is actually a binary stream (do you really want to spend your precious money on a LOB repository?)
- XML is the way to go
  - Period.

Why WebDAV?

WebDAV is cool!

- It’s standard
- It’s hierarchical
- It’s networked
- It’s extensible
- It’s exchangeable

– It’s ubiquitous

– It’s XML

– It’s HTTP

– It’s underhyped (until now)

Why not WebDAV?



WebDAV sucks!

- Transport intermixed with payload
- (some) Standards are not well defined
- Poor, if any, real XML support in metadata
- Poor client support (mostly just filesystem like operations)
- Clumsy for hard tasks

But in the end, it just sucks less

Apache Cocoon: the Apache XML framework

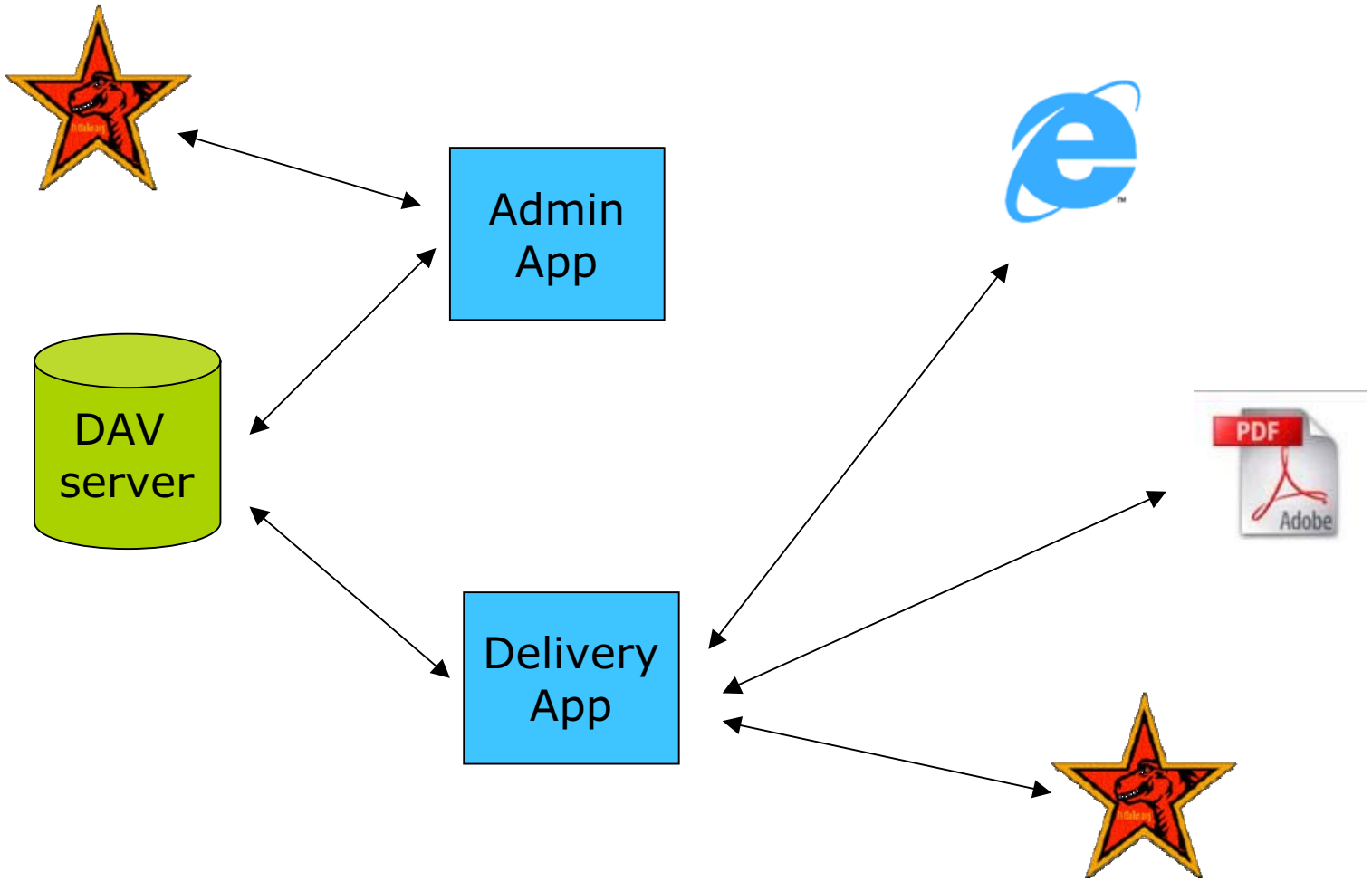
- If you're serious with XML, you should consider Cocoon
- Flexible, extensible, easy way to deal with XML
- Pipeline model
- Access to many (many!) datasources
- Please follow the other Cocoon talks for more

xml business alliance
OrixO
the xml business alliance

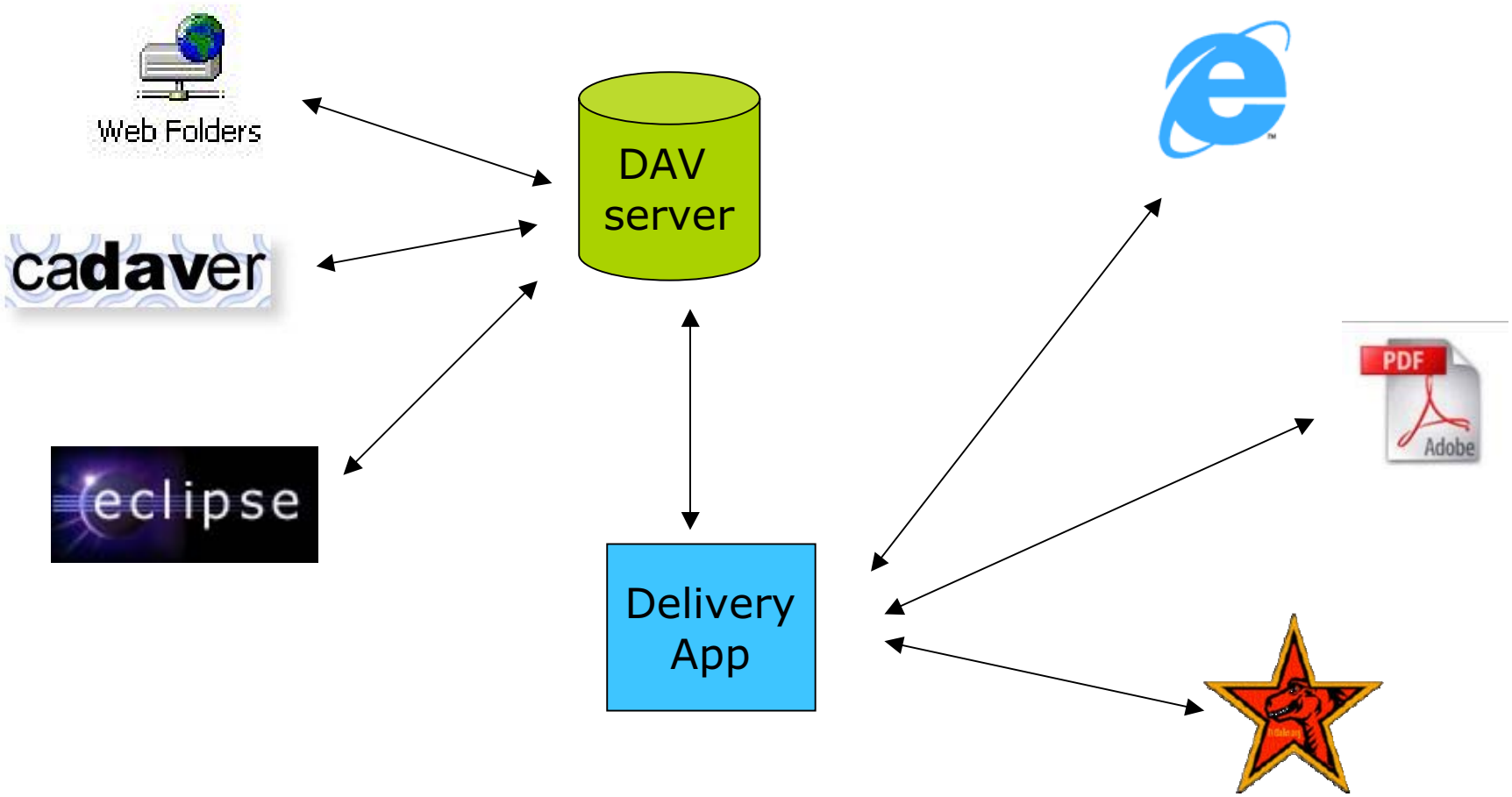
(yet another) Dynamic Duo?

- WebDAV is HTTP + XML
- Cocoon is (also) a servlet dealing with XML
- Full WebDAV support (some experimental):
  - Client (stable)
  - Server (needs work)
  - Proxy (really promising)
- Understands the DASL protocol
- WebDAV is a Source (think Jakarta VFS or java.io.File)

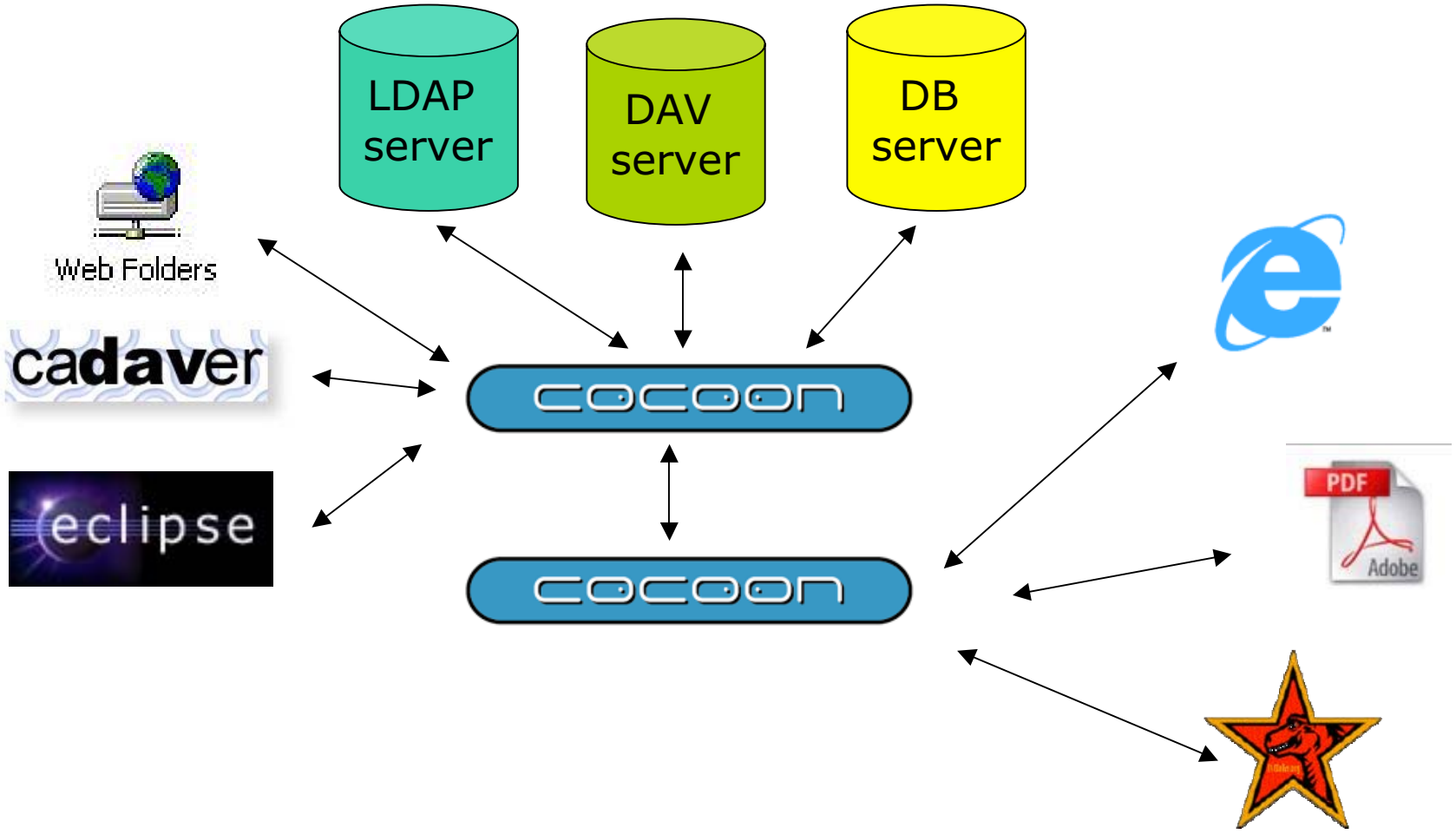
Cocoon/WebDAV architectures



Cocoon/WebDAV architectures



Cocoon/WebDAV architectures



Why a WebDAV proxy?



More flexibility

–Add virtual resources (e.g. a PDF view or resized images)

–Perform tasks upon WebDAV events (send an email when a file is changed)

–Provide easy, pluggable and effective authentication

–Mangle properties using simple Explorer-like file managers

xml business alliance
OrixO
the xml business alliance

Solving performance issues: the classical way

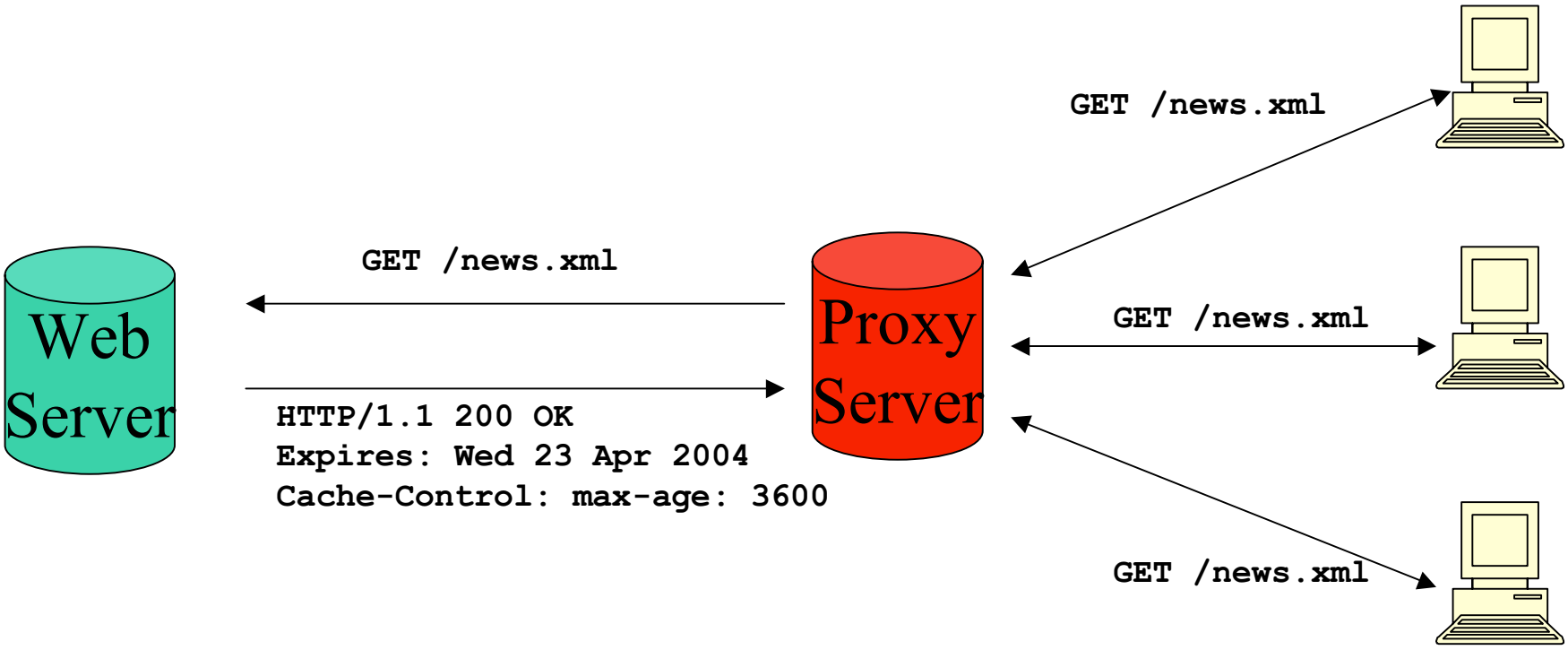
- Server Side Includes
- Static resources served by Apache (possibly with mod_rewrite hacks)
- More hardware
- Switch to static publication whenever possible



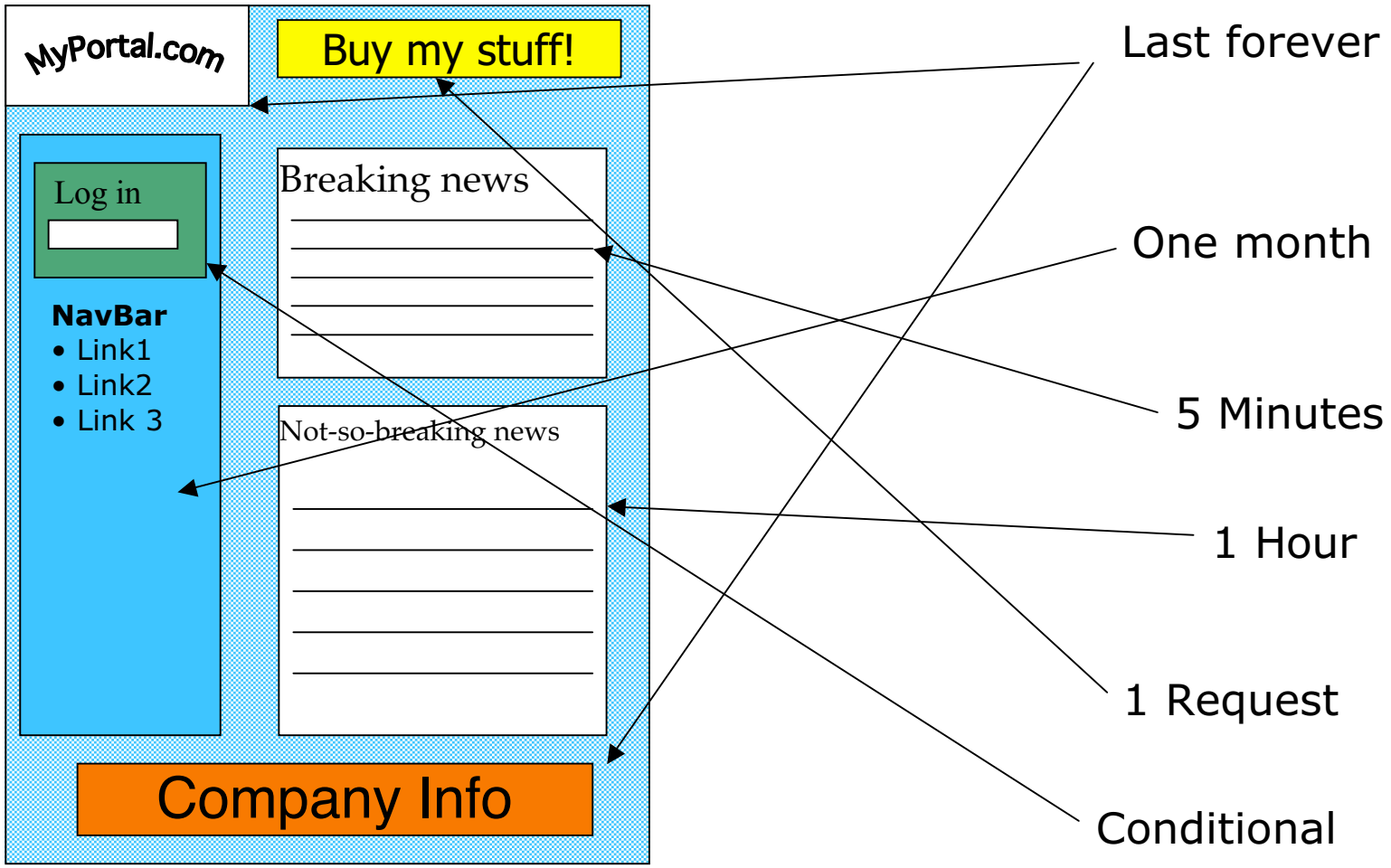
Edge Side Includes to the rescue

- Standard architecture (http://www.esi.org)
- Smarter includes on the reverse proxy side
- Can handle HTTP environment, conditionals and exceptions
- Supported in CDN (Content Delivery Networks)
- Supported by Squid 3.0

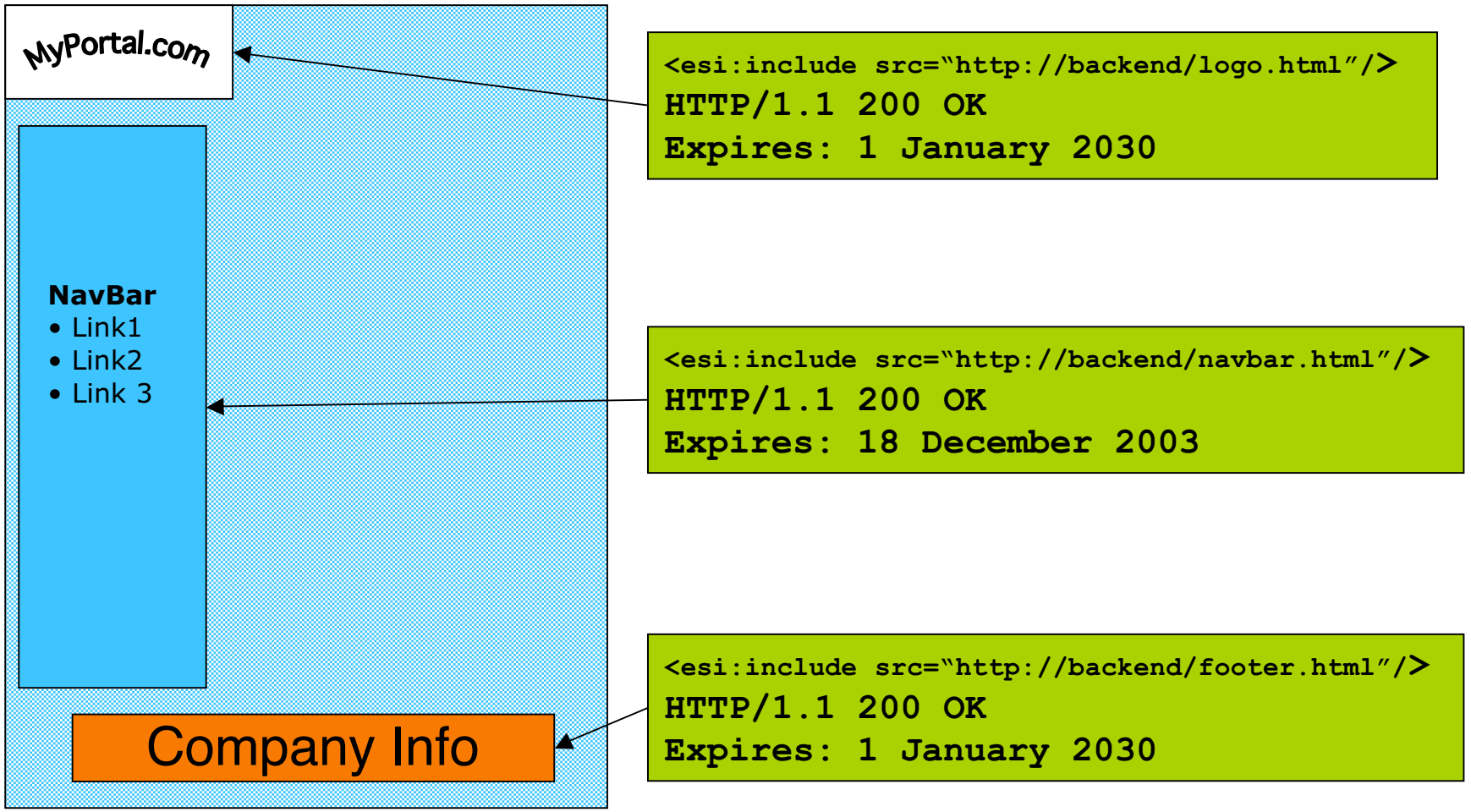
Reverse proxy architectures



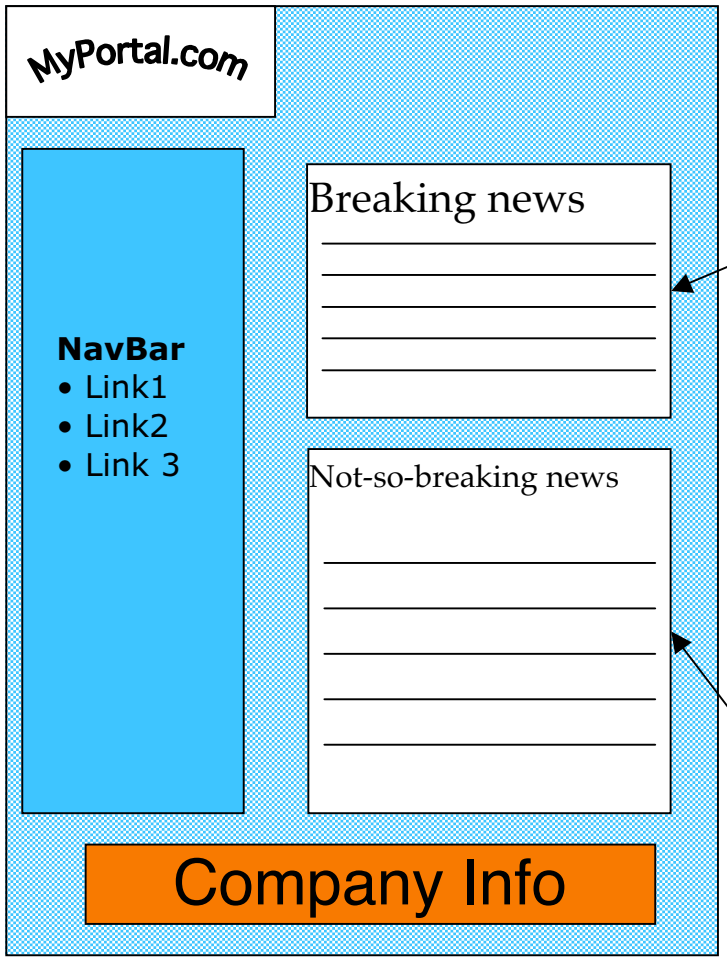
Life cycle of a Web page



Life cycle of a Web page



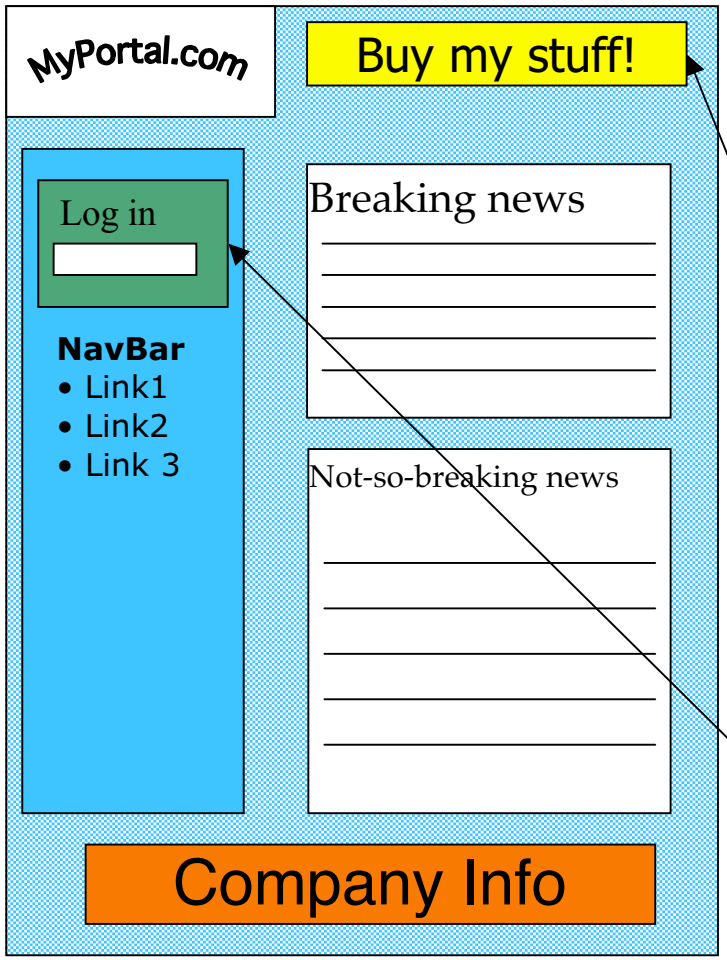
Life cycle of a Web page




```
<esi:try>
  <esi:attempt>
    <esi:include
src="http://backend/break.html"/>
  </esi:attempt>
  <esi:except>
    <esi:include
src="http://backend/latest.html"/>
  </esi:except>
<esi:try>
```

```
<esi:try>
  <esi:attempt>
    <esi:include
src="http://backend/news.html"/>
  </esi:attempt>
  <esi:except>
    <esi:include
src="http://backend/sorry.html"/>
  </esi:except>
<esi:try>
```

Life cycle of a Web page




```
<esi:try>
  <esi:attempt>
    <esi:include
src="http://adserver/ad.html"/>
  </esi:attempt>
  <esi:except>
    <esi:include
src="http://backend/default.html"/>
  </esi:except>
<esi:try>
```

```
<esi:choose>
  <esi:when
    test="$(HTTP_COOKIE{group})=='loggedin'">
    <esi:include
src="http://backend/welcome.xsp"/>
  </esi:attempt>
  <esi:except>
    <esi:include
src="http://backend/login.html"/>
  </esi:except>
<esi:try>
```



Issues to solve

- Keep the current backend architecture
- Store content on a WebDAV server
- Publish using Cocoon
- Ensure high availability and scalability

Content synchronization

xml business alliance
OrixO
the xml business alliance

Workflow:

- Recognize publishing events on the back end
- Pull content
- Sanitize it (convert to style-neutral XHTML)
- Store it on WebDAV

WebDAV repository



- Content is stored into hierarchies
- Media are on a separate tree
- Heavy use of WebDAV properties for publishing metadata
- DASL used extensively
  - ODBC support was added to Catacomb

Publishing

xml business alliance
OrixO
the xml business alliance

- Cocoon aggregates content from DAV and external resources (feeds)
- Uses DASL to decide on publishing, sequences, media and (some) style
- Content is ESI tagged and served through Squid
- Users are stored in LDAP

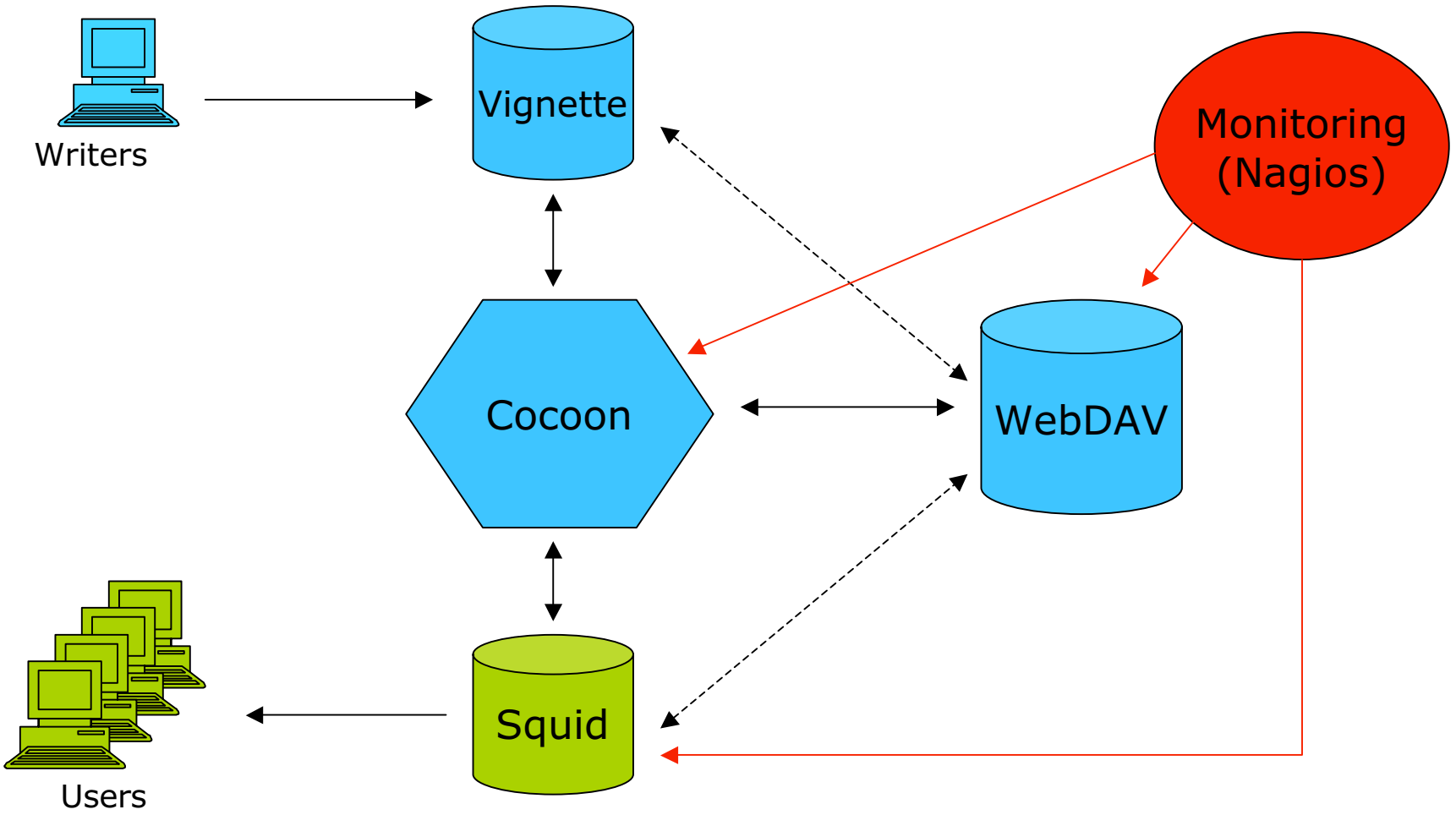
WebDAV properties

```
<?xml version="1.0" encoding="utf-8"?>
<D:multistatus xmlns:D="DAV:">
  <D:response xmlns:ns0="DAV:"
        xmlns:state="http://orixo.com/cms/state/1.0"
xmlns:content="http://orixo.com/cms/content/1.0">
    <D:href>/dav/article.xml</D:href>
    <D:propstat>
      <D:prop>
        <state:live>publish</state:live>
        <state:staging>draft</state:staging>
        <content:type>article</content:type>
      </D:prop>
      <D:status>HTTP/1.1 200 OK</D:status>
    </D:propstat>
  </D:response>
</D:multistatus>
```

DASL searches

```
<?xml version="1.0" encoding="utf-8"?>
<D:searchrequest xmlns:D="DAV:">
  <D:basicsearch>
    <D:select>
      <D:allprop/>
    </D:select>
    <D:from>
      <D:scope>
        <D:href>/dav/</D:href>
        <D:depth>infinity</D:depth>
      </D:scope>
    </D:from>
    <D:where>
      <D:and>
        <D:eq>
          <D:prop>
            <state:state xmlns:state="http://orixo.com/cms/state/1.0"/>
          </D:prop>
          <D:literal>publish</D:literal>
        </D:eq>
        <D:eq>
          <D:prop>
            <content:type xmlns:content="http://orixo.com/cms/content/1.0"/>
          </D:prop>
          <D:literal>article</D:literal>
        </D:eq>
      </D:and>
    </D:where>
  </D:basicsearch>
</D:searchrequest>
```

Putting it all together





Where to go from here

- Mix ‘n match WebDAV & JSR 170
- Finish up ODBC Catacomb porting
- Provide a flexible UI for the backend (XUL? Twingle?)
- Solve the XML editing issue
- Implement ESI in Apache

Questions?

This presenation has ended.
Click anywhere to return.

_

